

Domain Driven Design

Building Microservice Systems

Sharif University of Technology, Spring 2026

Ali Abrishami

Seyyed Hossein Mirhosseini



Design Heuristics



It Depends!

- “**It depends**” is the correct answer to almost any question in software engineering, but **not** really practical.
- We will explore what “it” depends on.
- This section bridges strategic design decisions and tactical design patterns.
- We will learn heuristics for applying analysis tools to drive various software design decisions: that is, (business) domain-driven (software) design.



Heuristic

- A heuristic is not a hard rule that is guaranteed and mathematically proven to be correct in 100% of cases.
- Rather, it's a **rule of thumb**: not guaranteed to be perfect, yet sufficient for one's **immediate** goals.

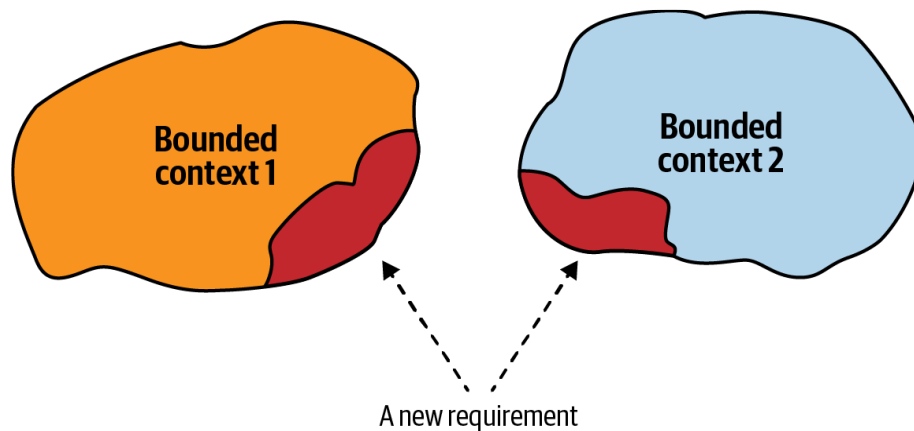


Designing Bounded Contexts

- *What is the optimal size of a bounded context?*
- There are many useful and revealing heuristics for defining the boundaries of a service. Size is one of the **least useful**.
- Software changes affecting multiple bounded contexts are expensive and require **lots of coordination**, especially if the affected bounded contexts are implemented by **different teams**.
- Refactoring bounded context boundaries is an **expensive** undertaking, and the ineffective boundaries remain unattended and end up accumulating **technical debt**.



Designing Bounded Contexts (cont.)



A change affecting multiple bounded contexts



Designing Bounded Contexts (cont.)

- Broad bounded context boundaries, or those that encompass multiple subdomains, make it **safer to be wrong**.
- When designing bounded contexts, start with **wider** boundaries.
- If required, decompose the wide boundaries into smaller ones as you gain **domain knowledge**.
- When creating a bounded context that contains a core subdomain, you can **protect** yourself against unforeseen changes by including other subdomains that the core subdomain interacts with most often.



Business Logic Implementation Patterns

- Both the **transaction script** and **active record** patterns are better suited for subdomains with simple business logic: **supporting subdomains** or integrating a third-party solution for a **generic subdomain**.
- The transaction script pattern can be used for simple data structures
- The active record pattern helps to encapsulate the mapping of complex data structures to the underlying database.
- The **domain model** and its variant, the event-sourced domain model, lend themselves to subdomains that have complex business logic: **core subdomains**.

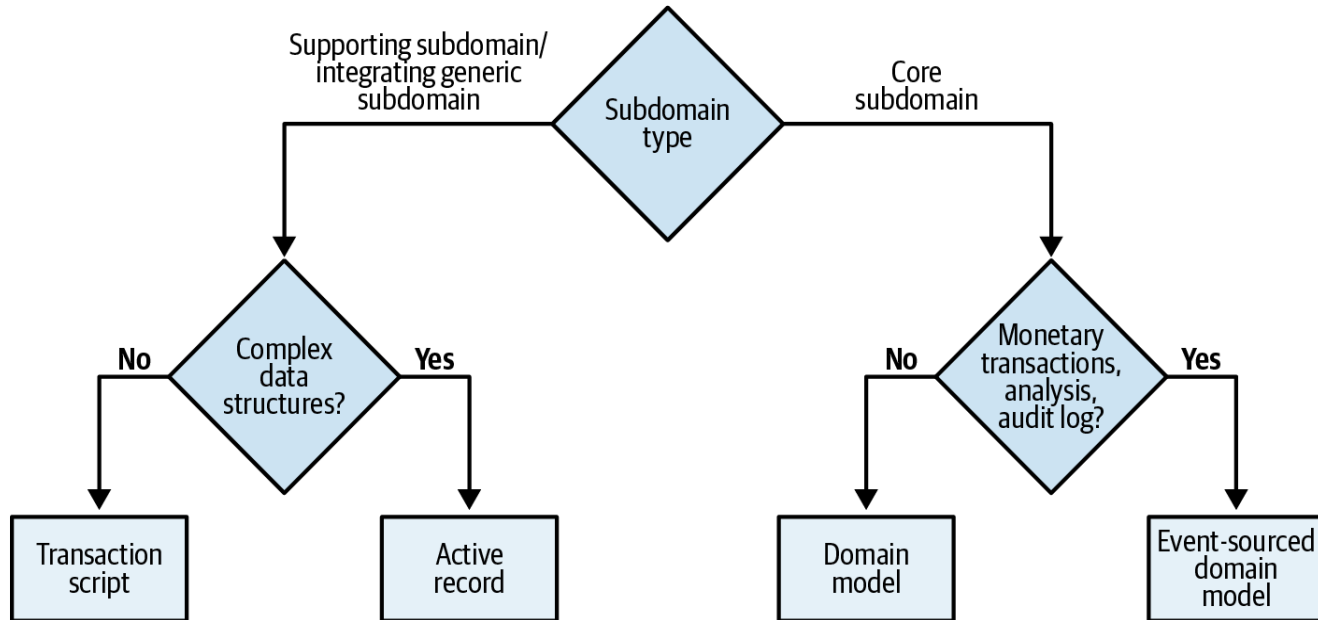


Business Logic Implementation Patterns (cont.)

1. Does the subdomain track **money** or other **monetary transactions** or have to provide a consistent audit log, or is deep analysis of its behavior required by the business? If so, use the ***event-sourced domain model***. Otherwise...
2. Is the subdomain's **business logic complex**? If so, implement a ***domain model***. Otherwise...
3. Does the subdomain include **complex data structures**? If so, use the ***active record*** pattern. Otherwise...
4. Implement a ***transaction script***.



Business Logic Implementation Patterns (cont.)



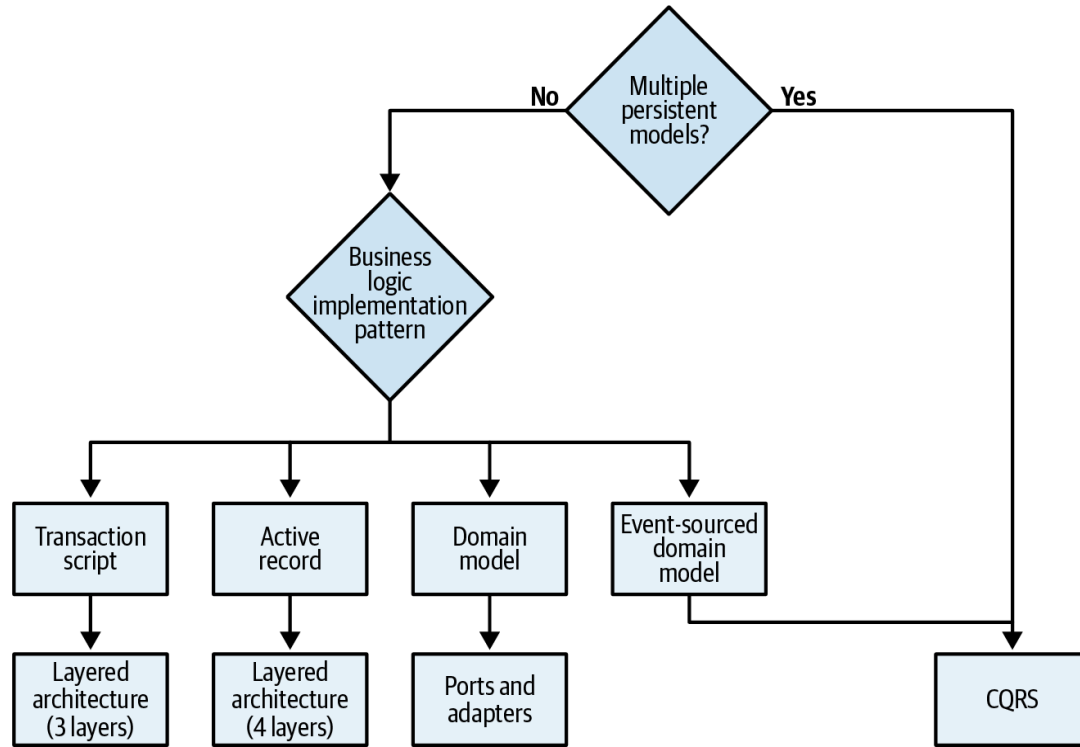
Decision tree for business logic implementation pattern



Architectural Patterns

- The event-sourced domain model requires **CQRS**. Otherwise, the system will be extremely limited in its data querying options, fetching a single instance by its ID only.
- The domain model requires the **ports & adapters** architecture. Otherwise, the layered architecture makes it hard to make aggregates and value objects ignorant of persistence.
- The Active record pattern is best accompanied by a **layered** architecture with the additional application (**service**) **layer**. This is for the logic controlling the active records.
- The transaction script pattern can be implemented with a **minimal layered** architecture, consisting of only three layers.

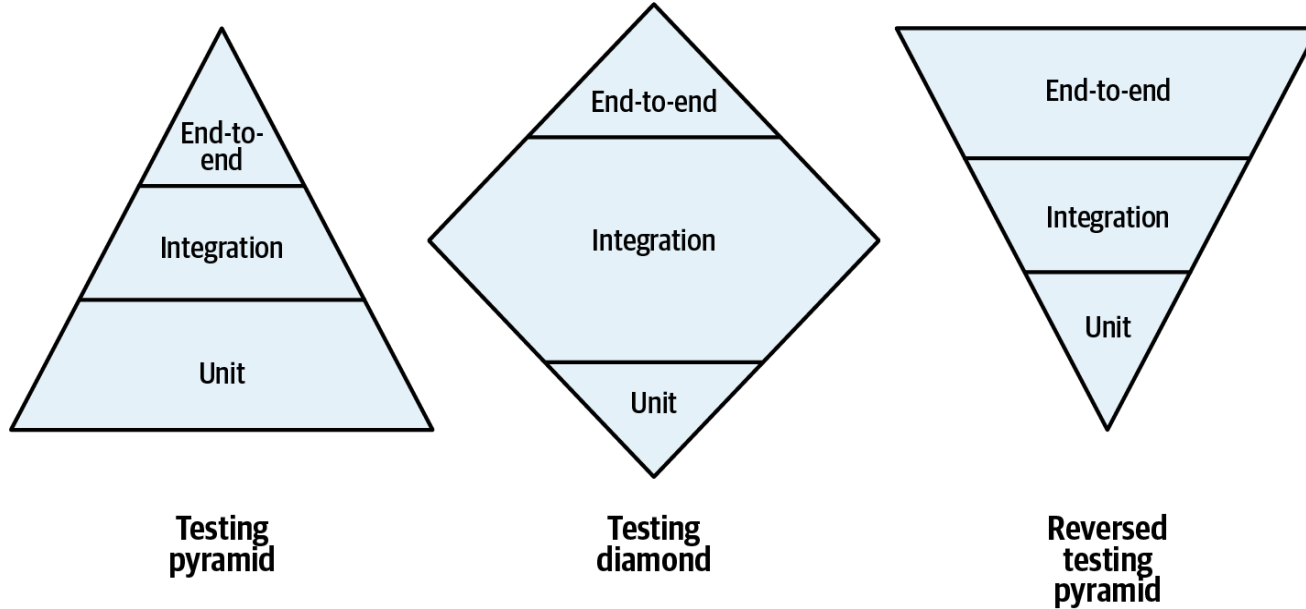




Architectural pattern decision tree



Testing Strategy



Testing strategies

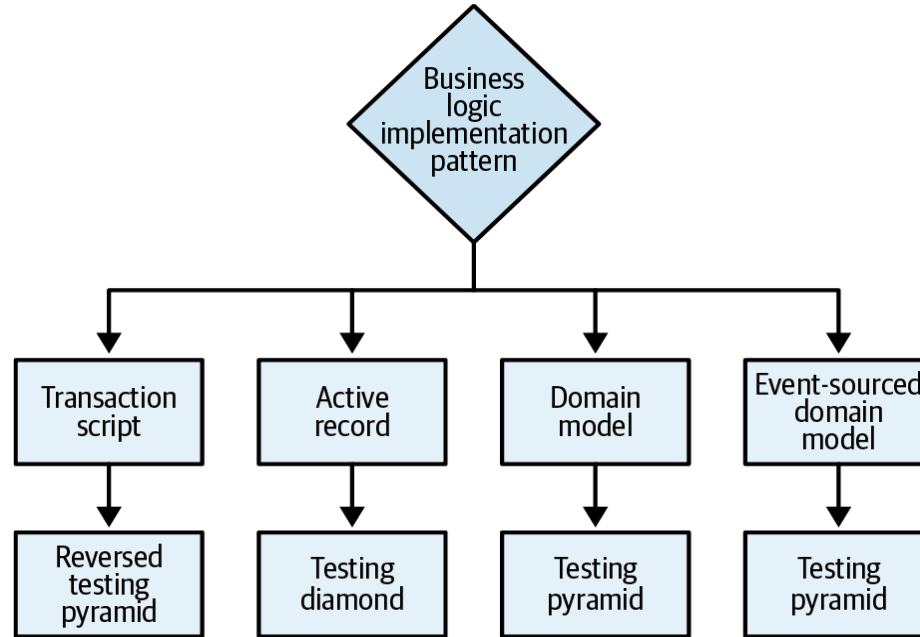


Testing Strategy (cont.)

- The difference between the testing strategies is **their emphasis** on the different types of tests: unit, integration, and end-to-end.
- Testing Pyramid
 - Emphasizes unit tests, fewer integration tests, and fewer end-to-end tests.
 - *Aggregates* and *value objects*.
- Testing Diamond
 - Focuses the most on integration tests.
 - *Active record* pattern.
- Reversed Testing Pyramid
 - Attributes the most attention to end-to-end tests.
 - *Transaction script* pattern.



Testing Strategy Decision Tree



Testing strategy decision tree



EventStorming



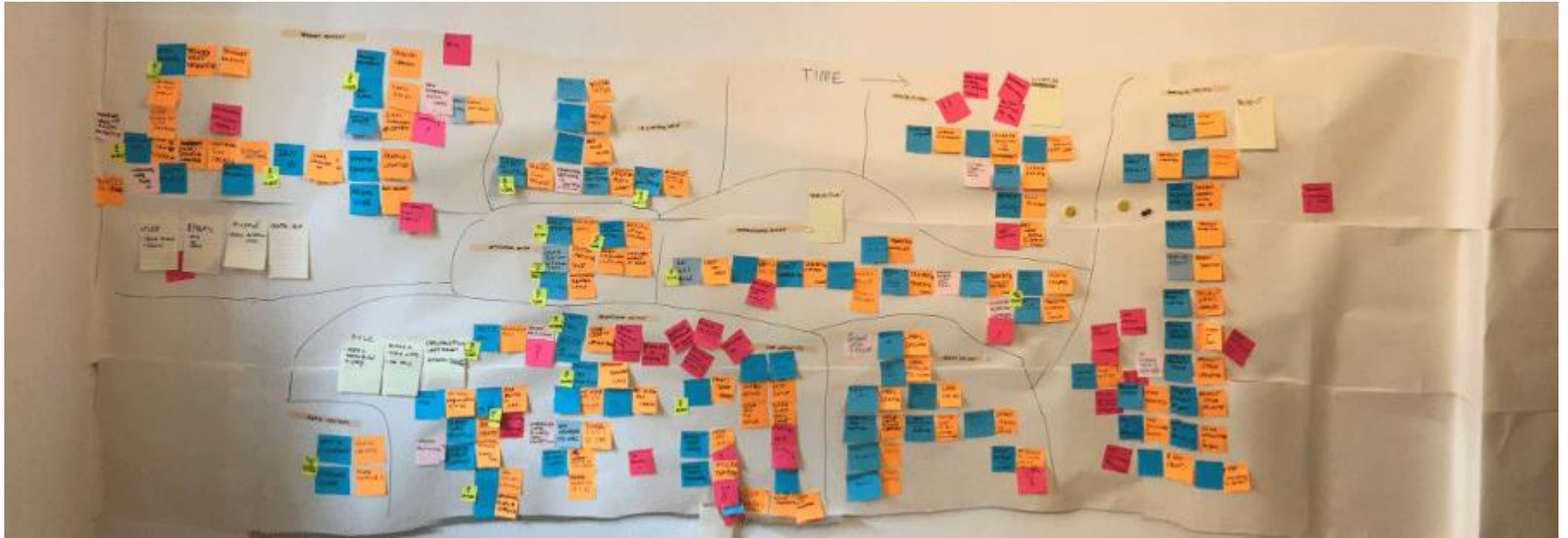
What Is EventStorming?

- EventStorming is a low-tech activity for a group of people to brainstorm and rapidly model a business process.
- In a sense, EventStorming is a tactical tool for **sharing business domain knowledge**.
- An EventStorming session has a **scope**: the **business process** that the group is interested in exploring.
- The participants are exploring the process as a series of domain events, represented by sticky notes, over a timeline.

**EVENT
STORMiNG**



EventStorming Output!



Who Should Participate in EventStorming?

- *Just keep in mind that the goal of the workshop is to **learn** as much as possible in the shortest time possible. We invite key people to the workshop, and we don't want to **waste** their valuable time.*
 - Alberto Brandolini, creator of the *EventStorming* workshop
- Anyone related to the business domain in question can participate:
 - Engineers,
 - Domain experts,
 - Product owners,
 - Testers,
 - UI/UX designers,
 - Support personnel,
 - And so on.



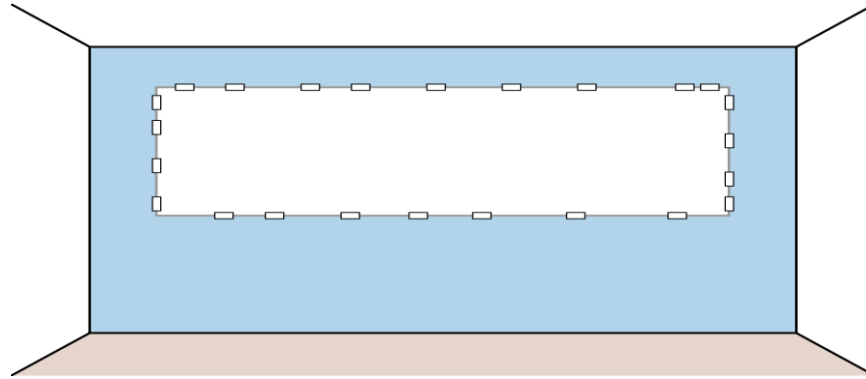
What Do You Need for EventStorming?

- *Modeling space*
 - First, you need a large modeling space. A whole wall covered with butcher paper makes the best modeling space.
- *Sticky notes*
 - Next, you need lots of sticky notes of different colors. The notes will be used to represent different concepts of the business domain.
- *Markers*
 - You'll also need markers that you can use to write on the sticky notes.
- *Snacks*
 - A typical EventStorming session lasts about two to four hours.



The EventStorming Process

- An EventStorming workshop is usually conducted in 10 steps. During each step, the model is enriched with additional information and concepts.



Modeling space for EventStorming

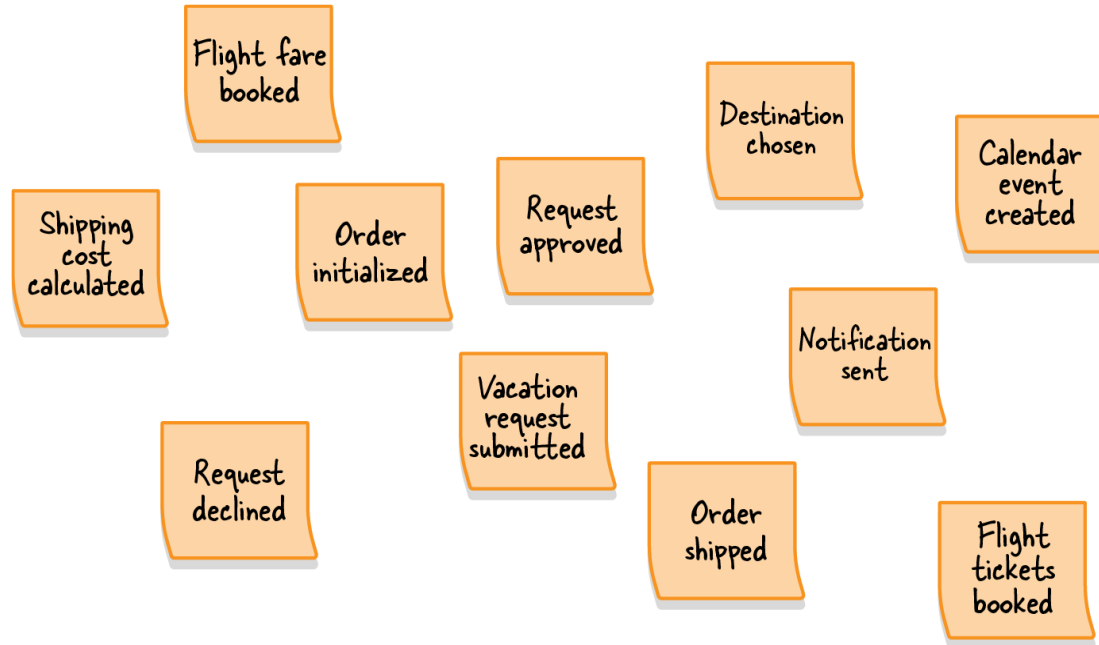


Step 1: Unstructured Exploration

- EventStorming starts with a **brainstorm** of the **domain events** related to the business domain being explored.
- A ***domain event*** is something interesting that has happened in the business.
- It's important to formulate domain events in the **past tense**.
- **Orange** sticky notes for modeling domain events.
- Brainstorming the possible things that can happen.



Step 1: Unstructured Exploration (cont.)

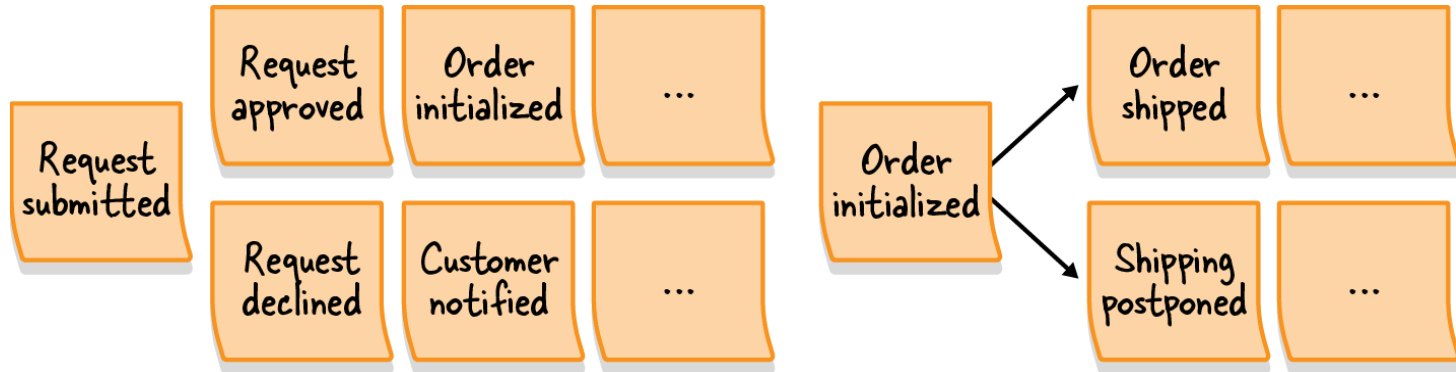


Step 2: Timelines

- Next, the participants go over the generated domain events and organize them in the **order** in which they occur in the business domain.
- The events should start with the “**happy path scenario**”:
 - The flow that describes a successful business scenario.
- Once the “happy path” is done, alternative scenarios can be added.
 - For example, paths where errors are encountered or different business decisions are taken.
- This step is also the time to fix incorrect events, remove duplicates, and of course, add missing events if necessary.



Step 2: Timelines (cont.)

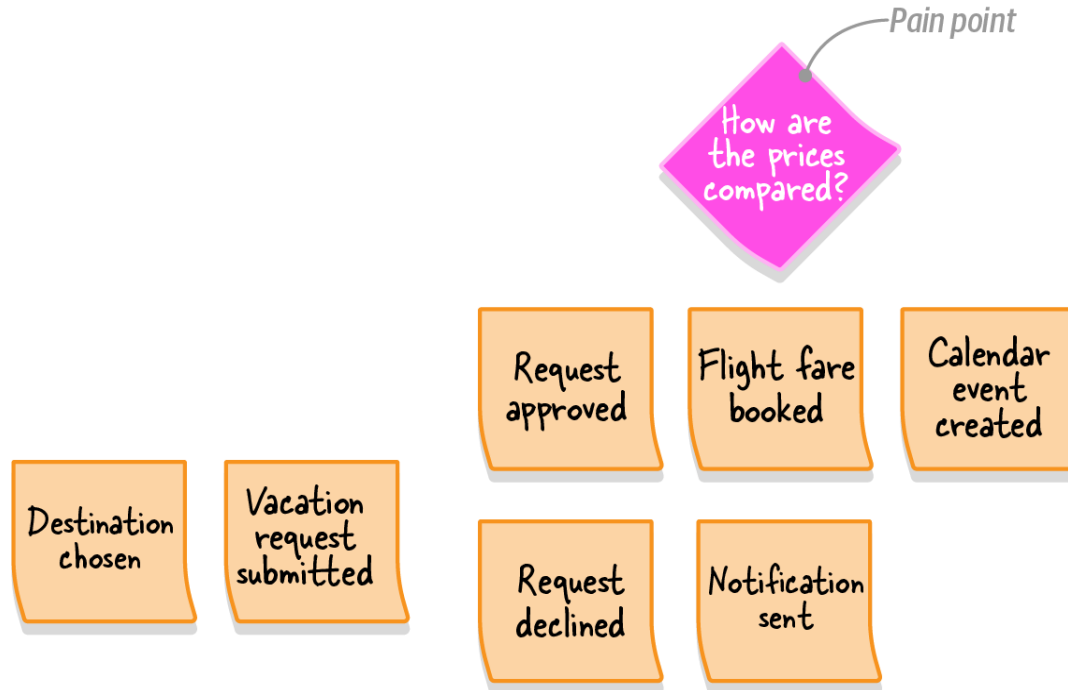


Step 3: Pain Points

- Once you have the events organized in a timeline, use this broad view to identify points in the process that **require attention**.
- These can be bottlenecks, manual steps that require automation, missing documentation, or missing domain knowledge.
- The pain points are marked with rotated (diamond) **pink** sticky notes,



Step 3: Pain Points (cont.)

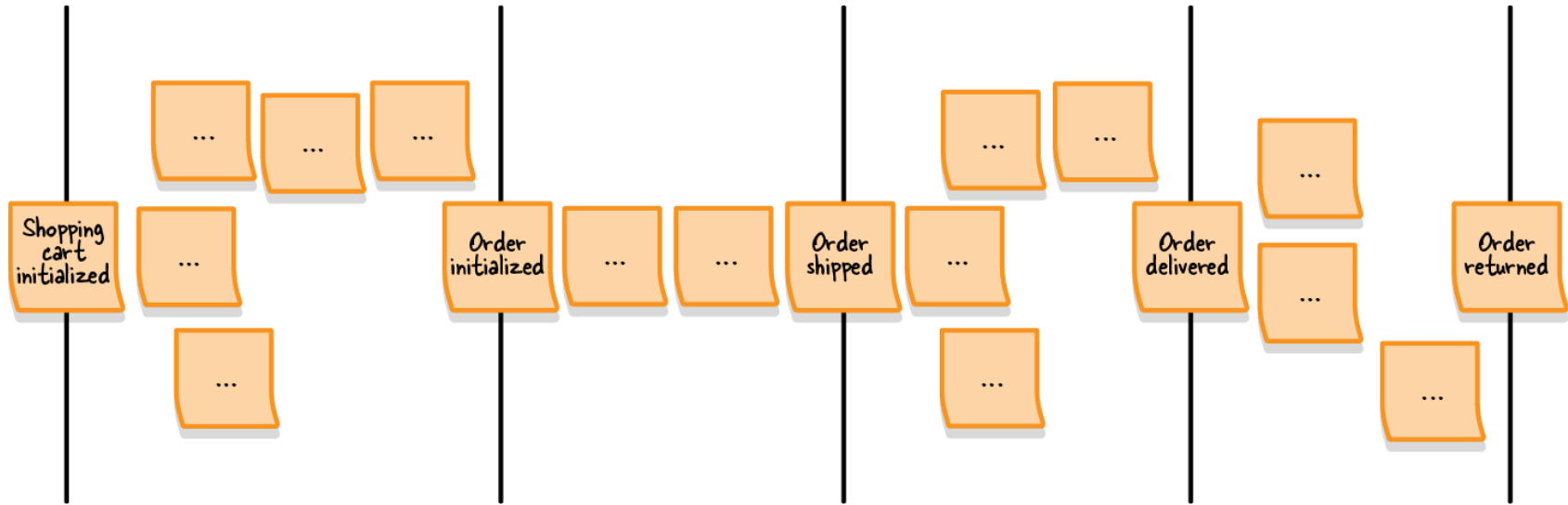


Step 4: Pivotal Events

- Once you have a timeline of events augmented with pain points, look for significant business events indicating a **change in context or phase**.
- These are called *pivotal events* and are marked with a **vertical bar** dividing the events before and after the pivotal event.
- Pivotal events are an indicator of **potential** bounded context boundaries.



Step 4: Pivotal Events (cont.)

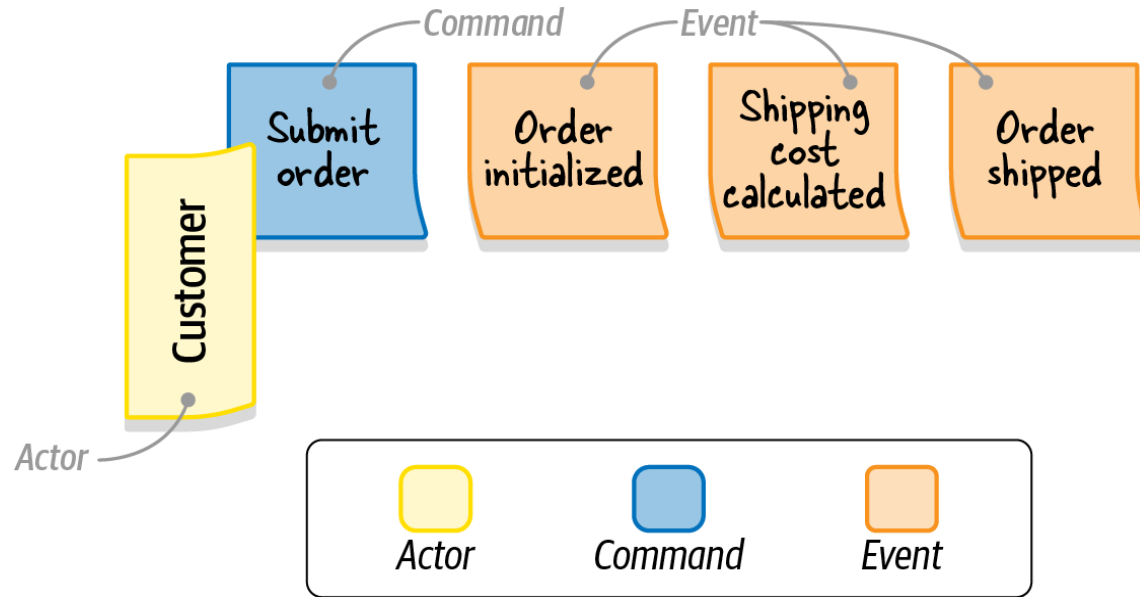


Step 5: Commands

- Whereas a domain event describes something that has already happened, a *command* describes **what triggered** the event or flow of events.
- Commands describe the system's **operations** and, contrary to domain events, are formulated in the **imperative**.
- Commands are written on **light blue sticky notes**.
- An actor in a specific role, the actor information is added to the command on a **small yellow sticky note**.



Step 5: Commands (cont.)

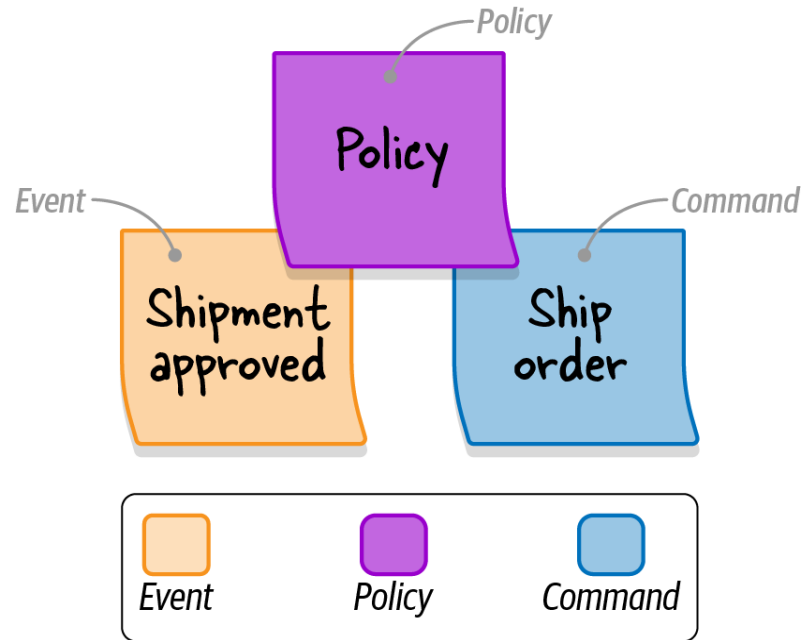


Step 6: Policies

- During this step, you look for automation policies that might execute those commands.
- An *automation policy* is a **scenario** in which **an event triggers the execution of a command**.
- In other words, a command is **automatically** executed when a specific domain event occurs.
- On the modeling surface, policies are represented as **purple sticky notes** connecting events to commands.



Step 6: Policies (cont.)

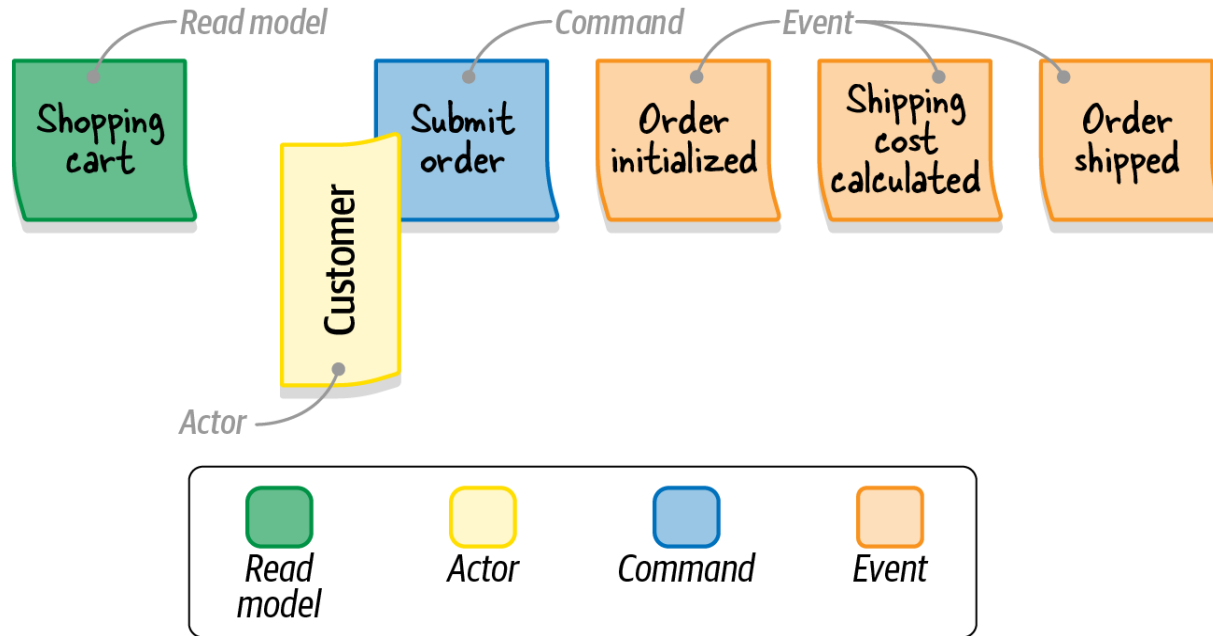


Step 7: Read Models

- A read model is the **view of data within the domain** that the actor uses to make a decision to execute a command.
- This can be one of the system's screens, a report, a notification, and so on.
- The read models are represented by **green sticky notes**.
- The read models are positioned before the commands.



Step 7: Read Models (cont.)



Step 8: External Systems

- This step is about augmenting the model with **external systems**.
- An *external system* is defined as any system that is **not a part of the domain** being explored.
- It can execute commands (input) or can be notified about events (output).
- The external systems are represented by **pink sticky notes**.

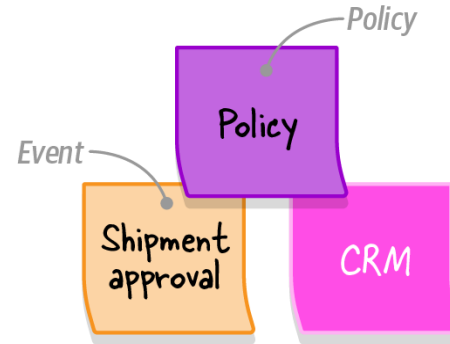


Step 8: External Systems (cont.)

An external system executes a command



Notifying an external system

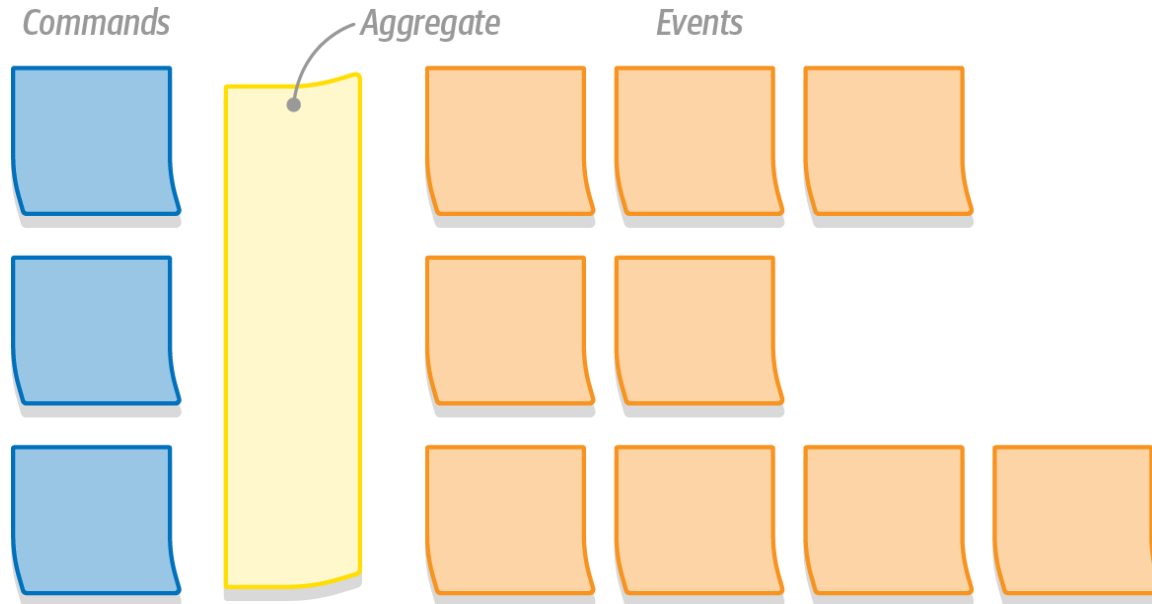


Step 9: Aggregates

- Once all the events and commands are represented, the participants can start thinking about **organizing related concepts** in **aggregates**.
- An *aggregate* **receives** commands and **produces** events.



Step 9: Aggregates (cont.)

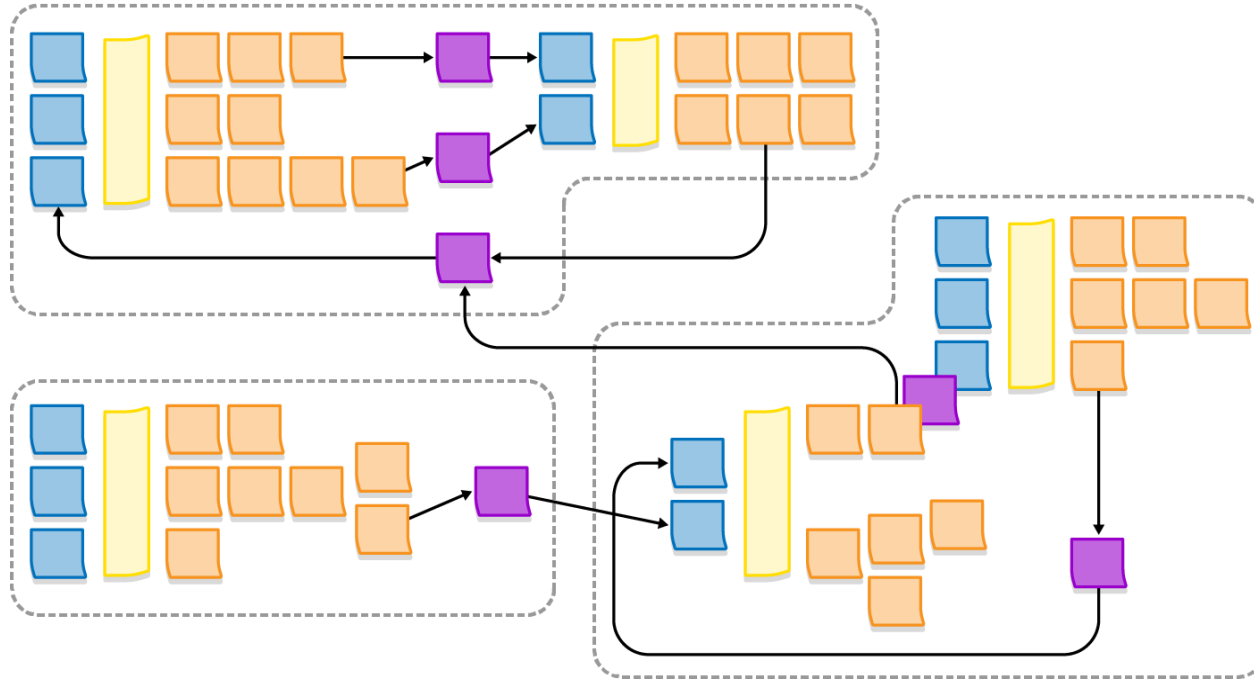


Step 10: Bounded Contexts

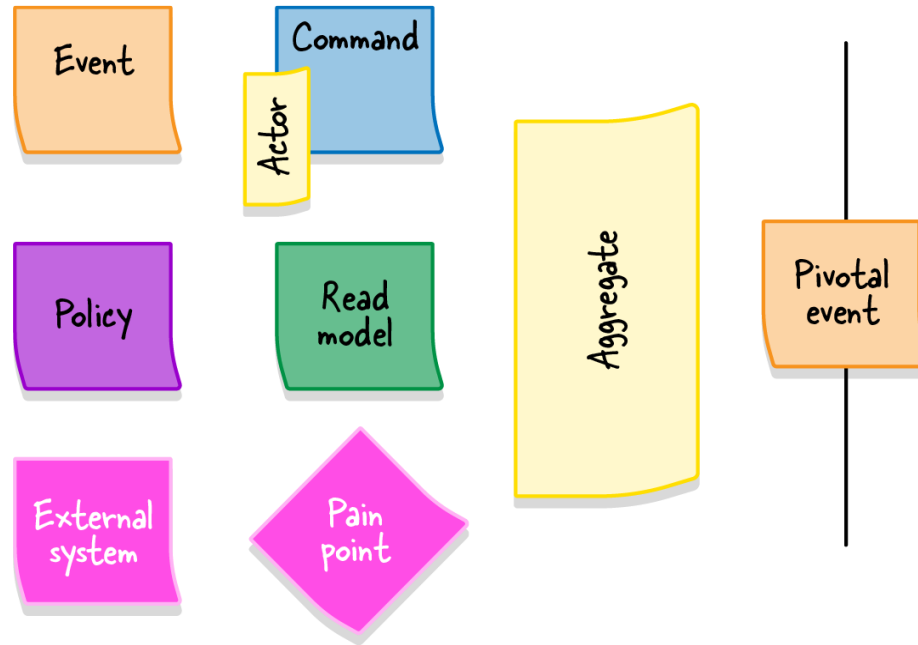
- The last step of an EventStorming session is to look for aggregates that are **related to each other**, either because they represent closely related functionality or because they're coupled through policies.



Step 10: Bounded Contexts (cont.)



Various Elements of the EventStorming Process



Variants

- Alberto Brandolini, the creator of the EventStorming workshop, defines the Event-Storming process as **guidance, not hard rules**.
- The resultant model covers a wide range of the company's business domain, builds a strong foundation for **ubiquitous languages**, and outlines possible **boundaries** for bounded contexts.
- At the end of a full EventStorming session, you will have a model describing the business domain's **events, commands, aggregates**, and even **possible bounded contexts**.
- However, all of these are just nice bonuses. The real value of an EventStorming session is the **process itself**—the **sharing of knowledge** among different stakeholders, alignment of their mental models of the business, discovery of conflicting models, and, last but not least, formulation of the **ubiquitous language**.



When to Use EventStorming

1. *Build a ubiquitous language*
2. *Model the business process*
3. *Explore new business requirements*
4. *Recover domain knowledge*
5. *Explore ways to improve an existing business process*
6. *Onboard new team members*



Remote EventStorming

- EventStorming was invented as a low-tech activity in which people interact and learn together in the same room.
- A number of tools attempted to enable collaboration and facilitation of **remote** EventStorming sessions.
- At the time of this writing, the most notable of them is **miro.com**.
- I prefer to limit online sessions to **five** participants.



DDD and Microservices



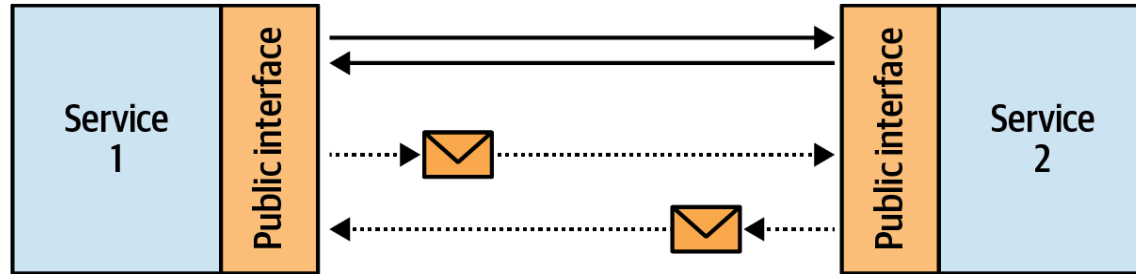
Microservices

- In the mid-2010s, microservices took the software engineering industry by **storm**.
- The intent was to address modern systems' need to **change quickly, scale**, and fit the distributed nature of **cloud computing** naturally.
- Historically, microservices are often associated with DDD, especially with the **bounded context pattern**.
- how to leverage DDD to design effective microservices-based systems.



What Is a Service?

- A service is a mechanism that enables access to one or more capabilities, where the access is provided using a **prescribed interface**.
 - **Synchronous**: such as a request/response model.
 - **Asynchronous**: such as a model that is producing and consuming events.

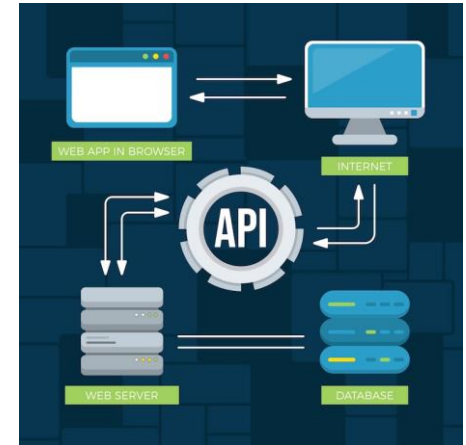


Communication between services



Service Interface

- A service's public interface defines the service itself: the **functionality** exposed by the service.
- A well-expressed interface is enough to describe the functionality implemented by a service.



What Is a Microservice?

- Since a service is defined by its public interface, a microservice is a service with a **micro-public interface**: a micro-front door.
- Reducing a service's functionality also limits its reasons for change and makes the service more autonomous for development, management, and scale.
- Microservices **encapsulate** their databases. The data can only be accessed through a much more compact, integration-oriented public interface.



Microservice Goal

- The goal of the microservices architecture is to produce a **flexible** system.
- Hence, a system cannot be built out of independent components.
- In a proper microservices-based system, however decoupled, the services still have to be **integrated** and **communicate** with each other.



System Complexity

- In his book, *Composite/Structured Design*, Glenford J. Myers discusses how to structure procedural code to **reduce its complexity**.
- *There is much more to the subject of complexity than simply attempting to minimize the local complexity of each part of a program.*
*A much more important type of complexity is **global complexity**: the complexity of the **overall structure** of a program or system (i.e., the degree of association or interdependence among the major pieces of a program).*

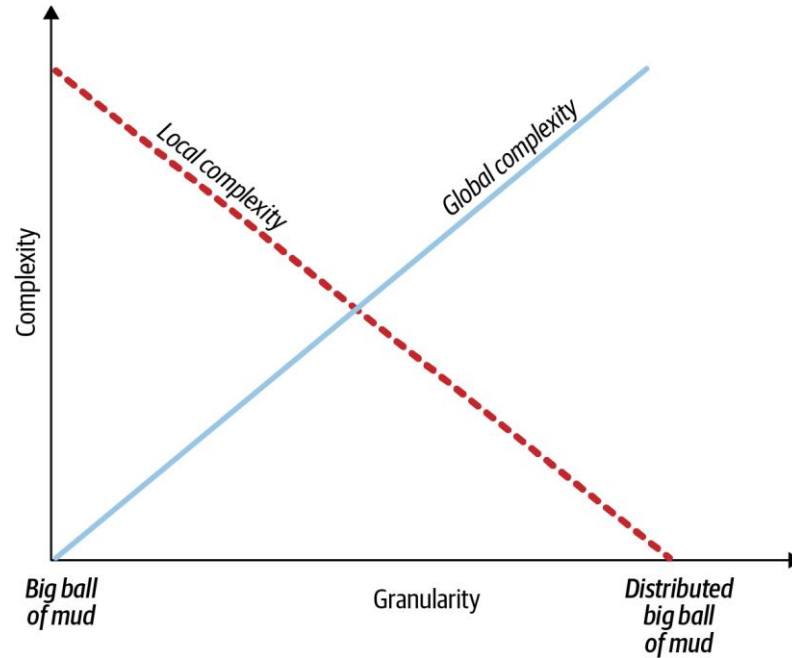


System Complexity (cont.)

- In our context, *local complexity* is the complexity of each **individual** microservice, whereas *global complexity* is the complexity of the **whole system**.
- It's surprisingly easy to reduce global complexity to a minimum.
- All we have to do is **eliminate** any interactions between the system's components—that is, implement all functionality in one monolithic service.
- To design a proper microservices-based system, we have to optimize both **global** and **local** complexities.



System Complexity (cont.)



Service granularity and system complexities



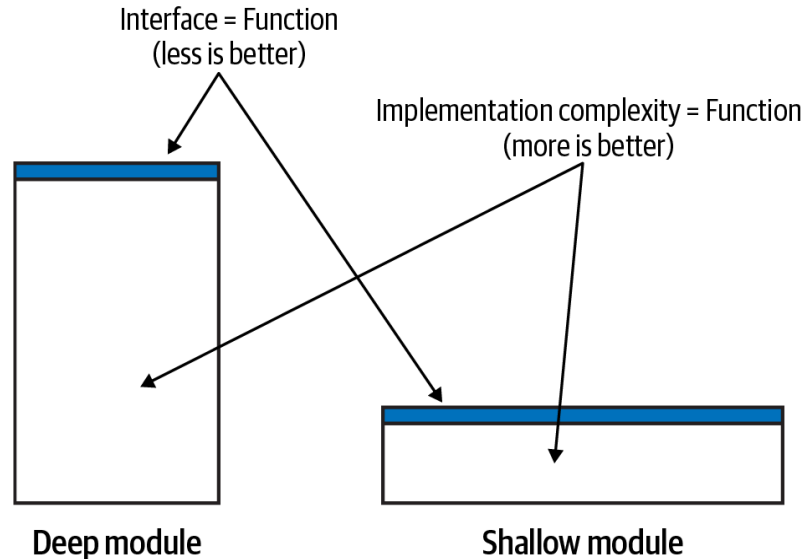
Microservices as Deep Services

- A **function** is what the module is supposed to do
 - Its business functionality.
- The **logic** is the module's business logic
 - How the module implements its business functionality.
- The rectangle's top edge represents the module's function, or the complexity of its public interface.
- A wider rectangle represents broader functionality, while a narrower one has a more restricted function and thus a simpler public interface.
- The area of the rectangle represents the module's logic, or the implementation of its functionality.



Deep Modules

- According to this model, **effective modules** are deep: a simple public interface encapsulates **complex logic**.

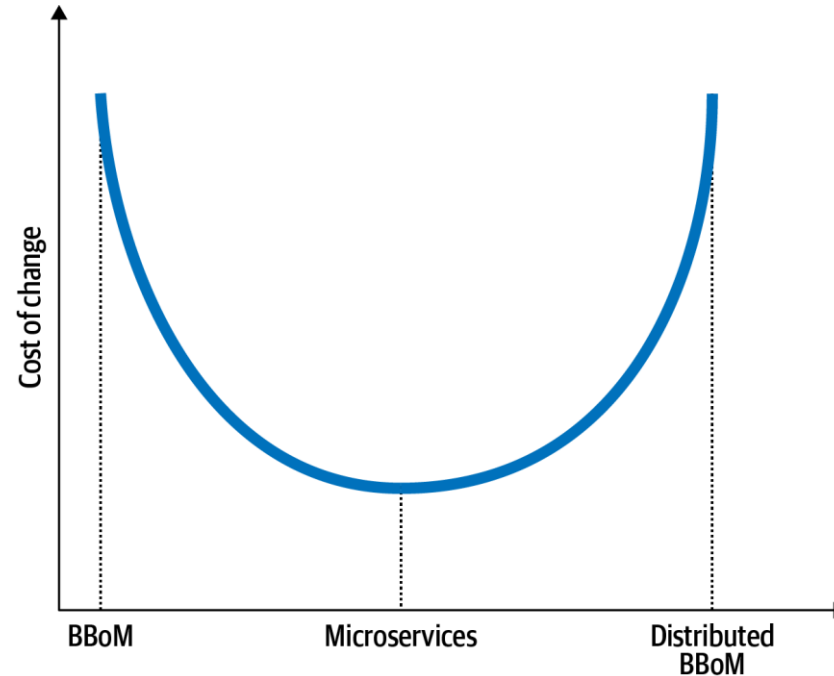


Microservices as Deep Modules

- From a system complexity standpoint, a deep module **reduces** the system's global complexity, while a shallow module **increases** it by introducing a component that **doesn't encapsulate its local complexity**.
- **Shallow services** are also the reason why so many microservices-oriented projects **fail**.
- The mistaken definitions of a microservice as a service having no more than X lines of code, or as a service that should be easier to rewrite than to modify, concentrate on the individual service while missing the most **important aspect of the architecture: the system**.



Granularity and cost of change



Granularity and cost of change



Domain-Driven Design and Microservices' Boundaries

- As microservices, many of the domain-driven design patterns discussed in the previous chapters are about **boundaries**:
 - The bounded context is the **boundary of a model**,
 - A subdomain **bounds a business capability**,
 - While aggregate and value objects are **transactional boundaries**.



Bounded Contexts and Microservices

- The microservices and bounded context patterns have a lot in common, so much so that the patterns are often **used interchangeably**.
- Both microservices and bounded contexts are **physical boundaries**.
- Microservices, as bounded contexts, are owned by **a single team**.



Bounded Contexts and Microservices (cont.)

- *Can we say that bounded contexts are microservices?*
- Bounded contexts protect the **consistency of ubiquitous languages** and models. No conflicting models can be implemented in the same bounded context.
- Therefore, the relationship between microservices and bounded contexts is **not symmetric**.
- Although microservices are bounded contexts, **not** every bounded context is a microservice.
- Bounded contexts, on the other hand, denote the boundaries of the largest valid monolith.

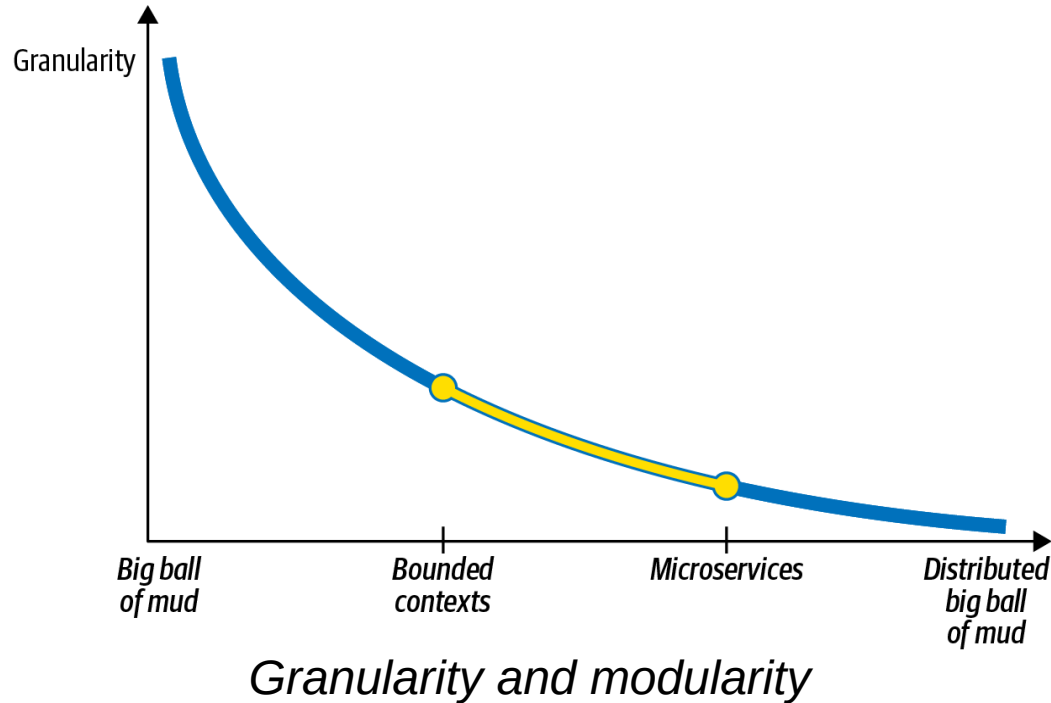


Bounded Contexts and Microservices (cont.)

- The figure demonstrates the relationship between bounded contexts and microservices.
- The area between the bounded contexts and microservices is safe. These are valid design options.
- However, if the system is not decomposed into proper bounded contexts or is decomposed past the microservices threshold, it will result in a **big ball of mud** or a **distributed big ball of mud**, respectively.



Bounded Contexts and Microservices (cont.)



Aggregates

- Decomposing an aggregate into multiple physical services, or bounded contexts, is not only suboptimal but leads to undesired consequences.

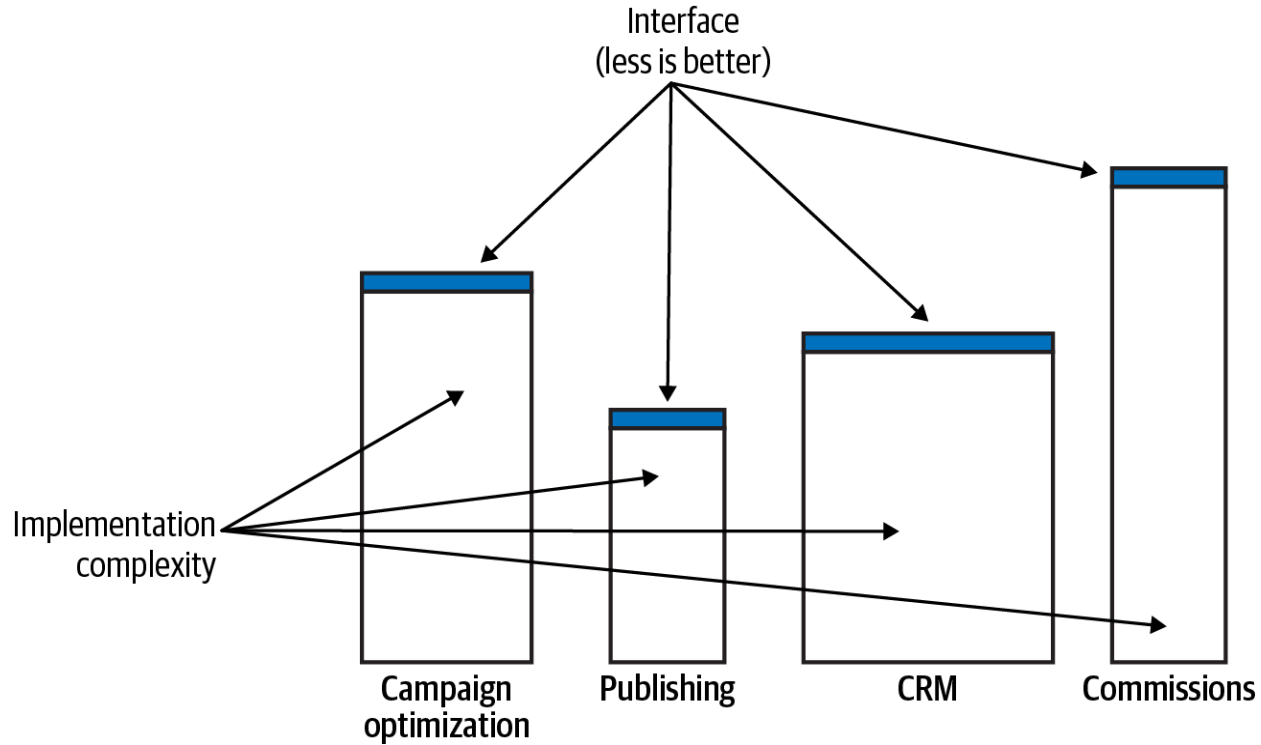


Subdomains

- A more balanced heuristic for designing microservices is to align the services with the **boundaries of business subdomains**.
- From a technical standpoint, subdomains represent sets of coherent use cases: using the same model of the business domain, working on the same or closely related data, and having a strong functional relationship.



Subdomains (cont.)



Subdomains (cont.)

- The subdomains' granularity and the focus on the functionality—the “what” rather than the “how”—makes subdomains naturally **deep modules**.
- A subdomain's description—the function—encapsulates the more complex implementation details—the logic.
- The **coherent** nature of the use cases contained in a subdomain also ensures the resultant module's depth.
- Splitting them apart in many cases would result in a more complex public interface and thus shallower modules.
- All of these things make subdomains a **safe boundary** for designing microservices.



References

- Learning Domain-Driven Design, *Vlad Khnonov*.
 - Chapter 10, 12, 14

